



DESENVOLUPAMENT WEB EN ENTORN SERVIDOR

Part 1: Conceptes bàsics de Python

RA's vinculades:

RA3: Escriu blocs de sentències embeguts en llenguatges de marques, seleccionant i utilitzant les estructures de programació

INTRODUCCIÓ A PYTHON

- Llenguatge de programació no tipat.
- Frameworks dedicats al backend
 - **Django**
 - **FastApi**
 - Flask
 - Pyramid
- Llibreries extenses dedicades a Machine learning, Big Data, Business Intelligence, etc
 - **matplotlib**
 - **Pandas**
 - SciPy
 - Seaborn
 - Polars
- Comunitat molt gran.
- Tot és un objecte.

INSTAL·LACIÓ I CONFIGURACIÓ

- **Linux (OPCIÓ MOLT RECOMANADA)**

Python ve instal·lat per defecte en gairebé tots els sistemes de Linux. Totes les imatges contingudes en aquest document estan extretes d'un ordinador amb el SO UBUNTU 22.04 de Linux.

Per saber la versió de Python instal·lada s'utilitza la següent comanda:

```
roger@roger-huawei:~/Documentos/github_repos/2024_2025/M7_24_25$ python3 --version  
Python 3.10.12
```

Per instal·lar Python3.10 seguir el tutorial d'instal·lació ➡

- **Windows**

Caldrà anar a la [pàgina oficial](#) de Python i descarregar-se l'instal·lador. Caldrà assegurar-se de marcar la opció *afegir python al PATH*.

Una cop hagi acabat la instal·lació amb la següent comanda al CMD hauria d'indicar la versió instal·lada:

CMD

```
#Comanda per saber la versió de Python instal·lada  
$ python3 --version  
Python 3.10.12
```

- **MacOS**

Per instal·lar python3 a MacOS caldrà tindre instal·lat el paquet d'instal·lació [Homebrew](#) que es troba a la web oficial i seguir els passos.

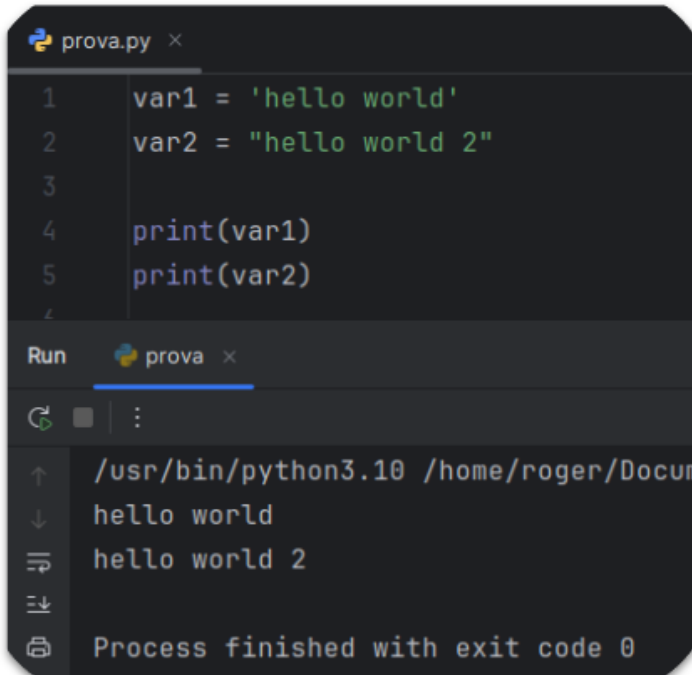
Un cop instal·lat Homebrew ja es pot instal·lar Python 3:

Terminal

```
#Comanda per instal·lar python a MacOS  
$ brew install python
```

VARIABLES I TIPUS DE DADES

Per crear una variable en python només cal assignar (=) un valor (**value**) a una paraula inventada (**nominventat**).



```
prova.py x
1  var1 = 'hello world'
2  var2 = "hello world 2"
3
4  print(var1)
5  print(var2)
6
Run  prova x
/usr/bin/python3.10 /home/roger/Docum
hello world
hello world 2
Process finished with exit code 0
```

En aquest exemple s'hi creen dues variables amb diferents valors (en aquest cas strings).

Amb la funció **print** s'imprimeix per consola allò que s'hi afegeixi dintre dels parèntesis.

(En blanc teniu el codi creat i en verd la resposta teòrica per terminal).

Fixar-se en les dues variables; a **var1** s'hi defineix un valor string (str) amb cometes dobles "" i en **var2** s'hi defineix un altre valor **str** amb cometes simples ". En qualsevol dels dos casos, Python ho treballa com **str**.

Tipus de dades

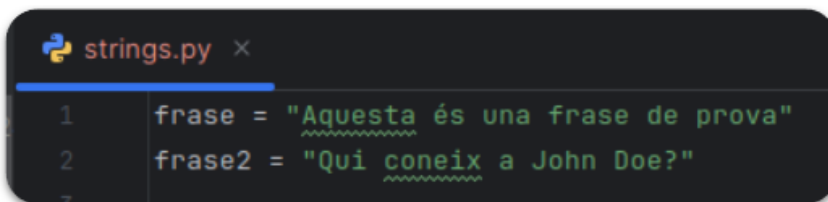
En qualsevol tecnologia (llenguatge) de programació s'hi trobaran els mateixos tipus de dades per treballar un programa. En la següent imatge s'hi exposen els més utilitzats:

```
# Declaració de variables de diferents tipus
var1 = 1 # Integer / int
var2 = "Gat" # String / str
var3 = 3.45 # Float
var4 = True # Booleano True o False
```

- **Strings**

Com ja s'ha vist en el primer exemple de la pàgina anterior, un **str** es pot expressar amb dobles cometes o amb cometes simples.

També s'hi pot treballar en un text on s'hi contingui els dos formats en un mateix **str**.



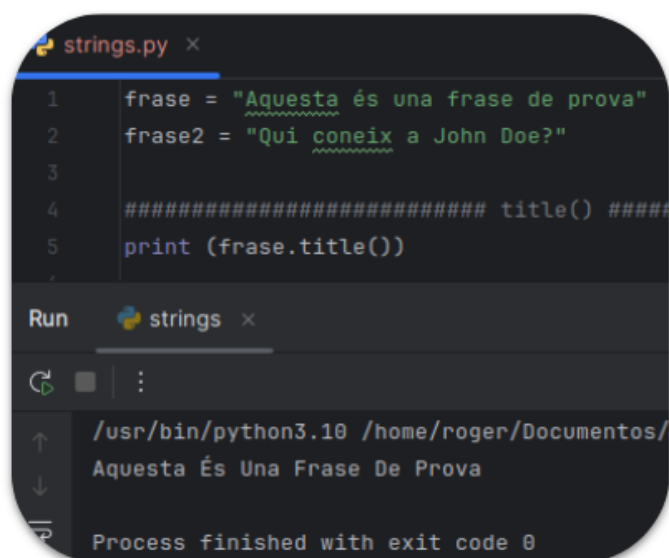
```
strings.py x
1 frase = "Aquesta és una frase de prova"
2 frase2 = "Qui coneix a John Doe?"
```

S'hi pot observar que en els dos strings s'hi barregen la utilització de cometes simples dintre de cometes dobles i de cometes dobles dintre de cometes simples

Com ja s'ha comentat a l'apartat **introducció a Python** tot és un objecte en python, per tant, un **String** és un objecte el qual se li poden aplicar certs mètodes ja creats per a treballar els valors de les variables.

- **title()**

El mètode **title()** permet posar la primera lletra, de cada paraula, en majúscules (independentment de si ja està o no en majúscules).



```
strings.py x
1 frase = "Aquesta és una frase de prova"
2 frase2 = "Qui coneix a John Doe?"
3
4 ##### title() #####
5 print (frase.title())

Run strings x

/usr/bin/python3.10 /home/roger/Documents/
Aquesta És Una Frase De Prova

Process finished with exit code 0
```

fonts a consultar:

- [enllaç 1](#)
- [enllaç 2](#)

En aquest exemple, la paraula emmagatzemada a **frase** és un string i hereda els mètodes de **String** i que pot utilitzar afegint el . a la paraula **frase**.

Provar en el vostre IDE, mirar què passa quan poseu **frase**. (amb el punt), veureu que surt una llista de mètodes a utilitzar. Aquests mètodes son heretats de l'objecte **String**.

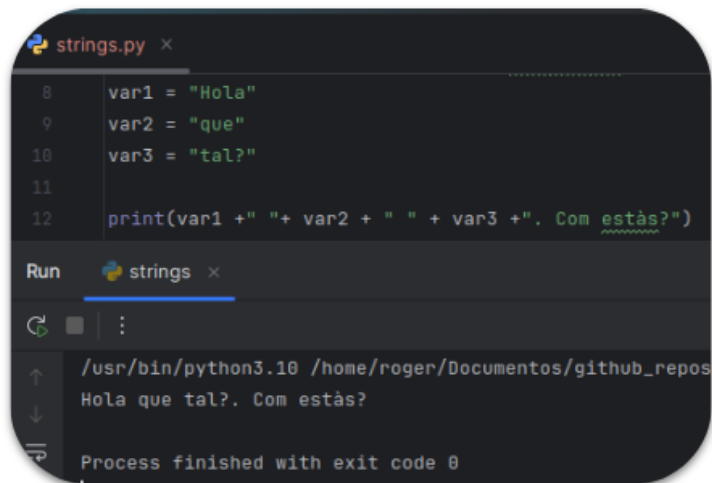
Seguint amb l'exemple, al posar **frase.title()** indica a Python que el mètode **title()** actúa sobre el valor de **frase**.

Altres mètodes a mirar-se:

- | | | | |
|---------------------------------------|---|---|---------------------------------------|
| ➤ <u>capitalize()</u> | ➤ <u>isdigit()</u> | ➤ <u>maketrans()</u> | ➤ <u>rstrip()</u> |
| ➤ <u>casefold()</u> | ➤ <u>isidentifier()</u> | ➤ <u>partition()</u> | ➤ <u>split()</u> |
| ➤ <u>center()</u> | ➤ <u>islower()</u> | ➤ <u>removeprefix()</u> | ➤ <u>splitlines()</u> |
| ➤ <u>count()</u> | ➤ <u>isnumeric()</u> | ➤ <u>removesuffix()</u> | ➤ <u>startswith()</u> |
| ➤ <u>encode()</u> | ➤ <u>isprintable()</u> | ➤ <u>replace()</u> | ➤ <u>strip()</u> |
| ➤ <u>endswith()</u> | ➤ <u>isspace()</u> | ➤ <u>rfind()</u> | ➤ <u>swapcase()</u> |
| ➤ <u>expandtabs()</u> | ➤ <u>istitle()</u> | ➤ <u>rindex()</u> | ➤ <u>title()</u> |
| ➤ <u>find()</u> | ➤ <u>isupper()</u> | ➤ <u>rjust()</u> | ➤ <u>translate()</u> |
| ➤ <u>format()</u> | ➤ <u>join()</u> | ➤ <u>rpartition()</u> | ➤ <u>upper()</u> |
| ➤ <u>formatmap()</u> | ➤ <u>ljust()</u> | ➤ <u>rsplit()</u> | ➤ <u>zfill()</u> |
| ➤ <u>index()</u> | ➤ <u>lower()</u> | | |
| ➤ <u>isalnum()</u> | ➤ <u>lstrip()</u> | | |
| ➤ <u>isalpha()</u> | | | |
| ➤ <u>isascii()</u> | | | |
| ➤ <u>isdecimal()</u> | | | |

- **Concatenar**

És necessari saber com ajuntar varis tipus de dades diferents i mostrar-ho com un string a l'usuari. En Python, el més usual és utilitzar el caràcter +.



```
strings.py x
8   var1 = "Hola"
9   var2 = "que"
10  var3 = "tal?"
11
12  print(var1 + " " + var2 + " " + var3 + ". Com estàs?")

Run strings x
/usr/bin/python3.10 /home/roger/Documentos/github_repos
Hola que tal?. Com estàs?
Process finished with exit code 0
```

CONDICIONAL

Un condicional ens serveix per escollir una resposta segons l'estat en el que estigui la variable avaluada.

```
condicionals.py x
8  b = 3
9  c = 5
10
11  if b < c:
12      print("b és més petit que c")
13  else:
14      print("c és més petiti que b")
15
16  # if b < c:
17  #     print(b + " és més gran que " + c)
18  # else:
19  #     print(c + " és més gran que " + b)
20
21  if b < c:
22      print(str(c) + " és més gran que " + str(b))
23  else:
24      print(str(b) + " és més gran que " + str(c))
25
26  d = 10
27  e = 12
28
29  if d > e:
30      print(f"{d} és més gran que {e}")
31  else:
32      print(f"{e} és més gran que {d}")
```

En aquest codi es pot observar que el funcionament del condicional depèn del valor de la variable que s'avalua.

Cal tenir en compte la tabulació dintre de condicional. Sense aquesta tabulació no s'executarà el codi correctament

```
b = 3
c = 5

if b < c:
    print("b és més petit que c")
else:
    print("c és més petiti que b")
```

BUCLES

Un bucle és aquell codi que es va repetint tantes vegades com el desenvolupador ho hagi indicat.

Hi ha dos maneres de fer un bucle, amb el **while** o amb **for**.

El **while** s'utilitza quan es vol anar seguint una seqüència fins que s'arribi a un límit indicat, mentre que el **for** s'utilitza gairebé sempre per recórrer arrays i/o matrius tant per extreure-hi dades com anar-hi afegint.

```
##### WHILE #####
i = 1
while i < 6:
    print(i)
    i += 1

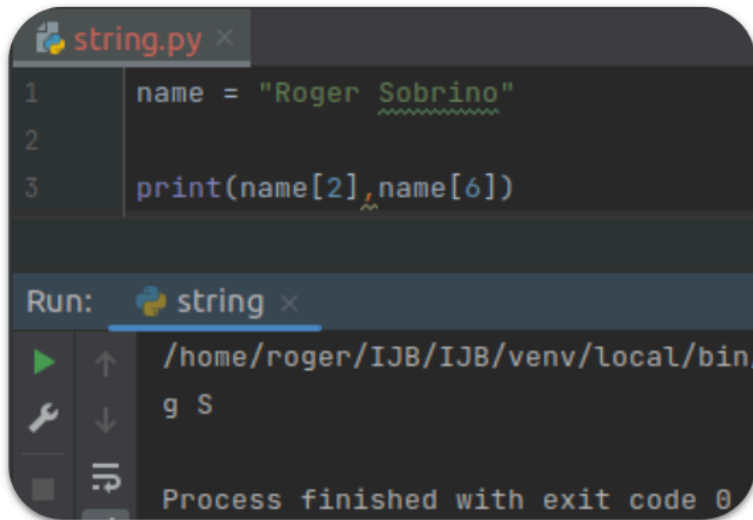
##### FOR #####
users = ["Roger", "Juanma", "J. Oriol"]
for x in users:
    print(x)
```

DATA TYPES

String

D'una variable que li assignem un valor com: **name = "Roger Sobrino"** se li pot:

- Extreure caràcters.

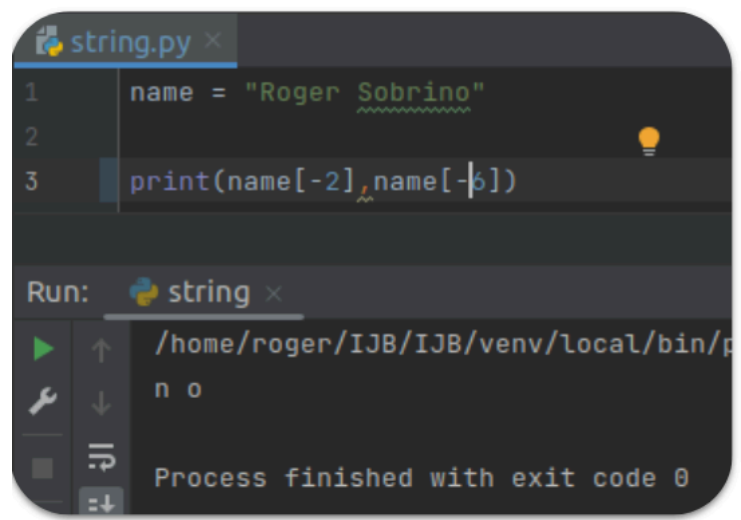


```
1 name = "Roger Sobrino"
2
3 print(name[2], name[6])
```

Run: string x

/home/roger/IJB/IJB/venv/local/bin/p
g S

Process finished with exit code 0

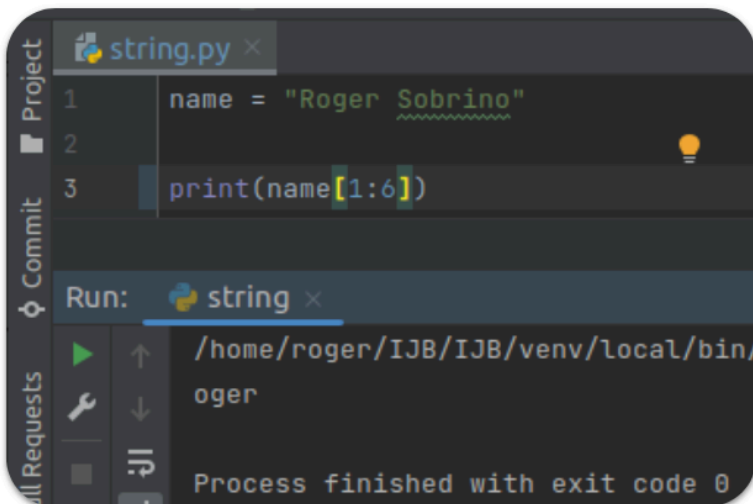


```
1 name = "Roger Sobrino"
2
3 print(name[-2], name[-6])
```

Run: string x

/home/roger/IJB/IJB/venv/local/bin/p
n o

Process finished with exit code 0



```
1 name = "Roger Sobrino"
2
3 print(name[1:6])
```

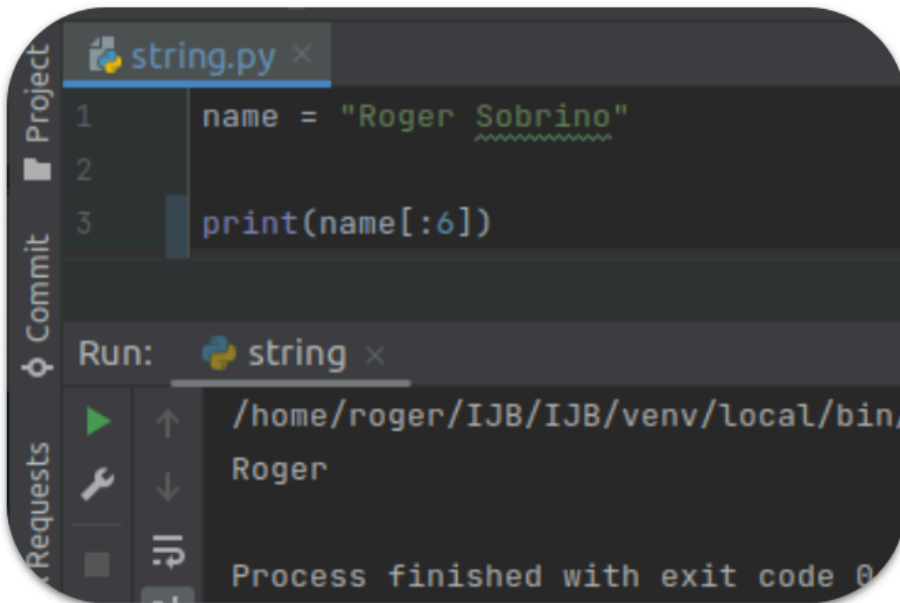
Run: string x

/home/roger/IJB/IJB/venv/local/bin/p
oger

Process finished with exit code 0

En aquest exemple es vol accedir
a les lletres **oger** de l'string

També es pot utilitzar l'operador **slice** (:) per indicar el rang que es busca.



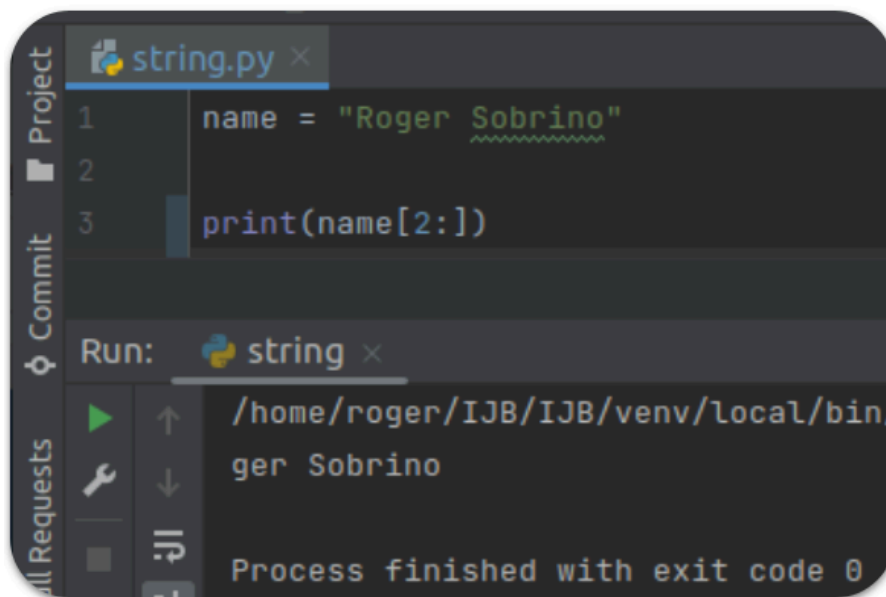
```
1 name = "Roger Sobrino"
2
3 print(name[:6])
```

Run: string

/home/roger/IJB/IJB/venv/local/bin/

Roger

Process finished with exit code 0



```
1 name = "Roger Sobrino"
2
3 print(name[2:])
```

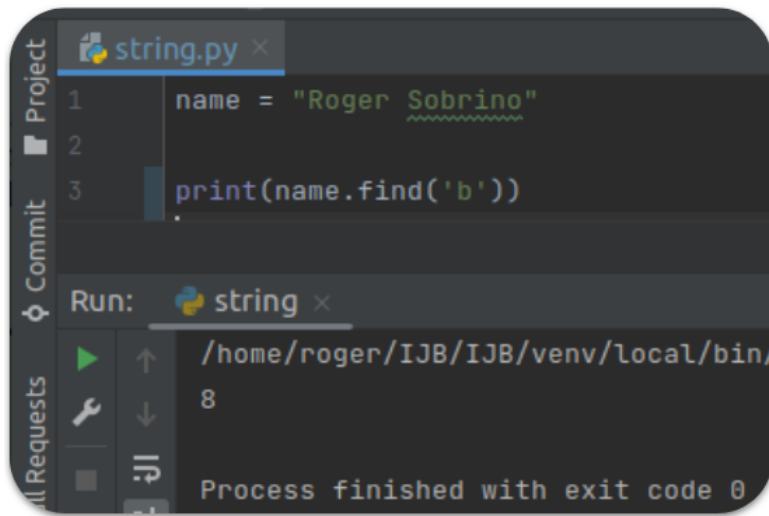
Run: string

/home/roger/IJB/IJB/venv/local/bin/

ger Sobrino

Process finished with exit code 0

- Trobar caràcters



The screenshot shows a code editor with a file named `string.py`. The code consists of three lines: `1 name = "Roger Sobrino"`, `2` (empty line), and `3 print(name.find('b'))`. Below the code editor, the 'Run' panel shows the command `/home/roger/IJB/IJB/venv/local/bin/` and the output `8`. At the bottom, it states 'Process finished with exit code 0'.

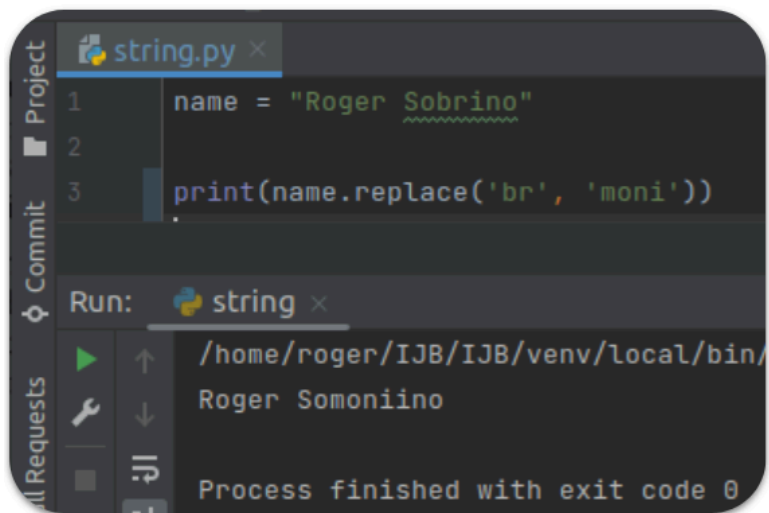
```
1 name = "Roger Sobrino"
2
3 print(name.find('b'))
```

Run: `string`

`/home/roger/IJB/IJB/venv/local/bin/`
`8`

Process finished with exit code 0

- Canviar text



The screenshot shows a code editor with a file named `string.py`. The code consists of three lines: `1 name = "Roger Sobrino"`, `2` (empty line), and `3 print(name.replace('br', 'moni'))`. Below the code editor, the 'Run' panel shows the command `/home/roger/IJB/IJB/venv/local/bin/` and the output `Roger Somoniino`. At the bottom, it states 'Process finished with exit code 0'.

```
1 name = "Roger Sobrino"
2
3 print(name.replace('br', 'moni'))
```

Run: `string`

`/home/roger/IJB/IJB/venv/local/bin/`
`Roger Somoniino`

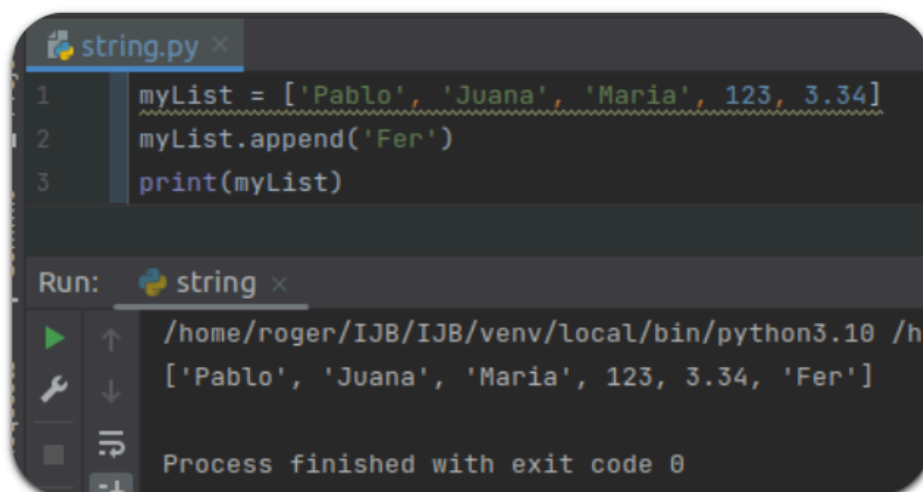
Process finished with exit code 0

List

Son col·leccions d'objectes de qualsevol tipus ordenades de forma arbitrària i sense tamany fixe. Mutables (es poden modificar) i son una seqüència de valors indexats per números enters.

```
myList = ['Pablo', 'Juana', 'Maria', 123, 3.34]
```

append(): Afegir un valor al final de la list.

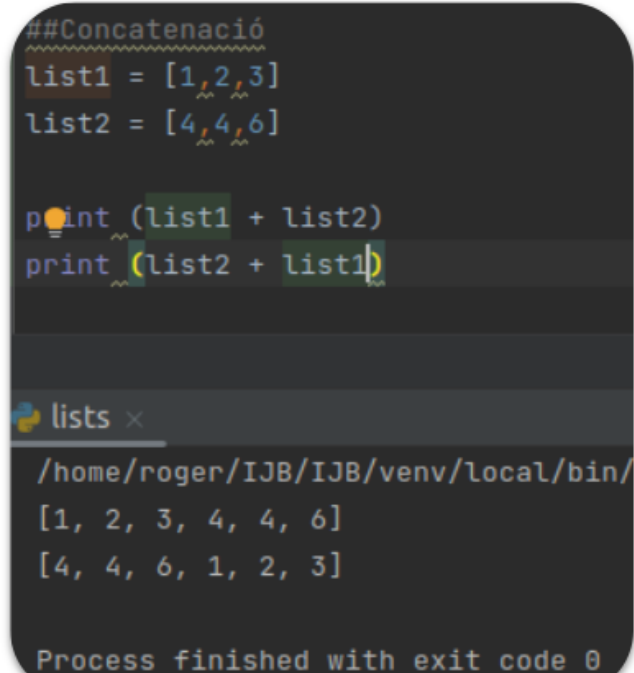


```
string.py x
1 myList = ['Pablo', 'Juana', 'Maria', 123, 3.34]
2 myList.append('Fer')
3 print(myList)

Run: string x
/home/roger/IJB/IJB/venv/local/bin/python3.10 /home/roger/IJB/IJB/venv/local/bin/python3.10 string.py
['Pablo', 'Juana', 'Maria', 123, 3.34, 'Fer']

Process finished with exit code 0
```

concatenar: unir varies llistes en una de sola amb el símbol +.



```
##Concatenació
list1 = [1, 2, 3]
list2 = [4, 4, 6]

print(list1 + list2)
print(list2 + list1)

lists x
/home/roger/IJB/IJB/venv/local/bin/python3.10 /home/roger/IJB/IJB/venv/local/bin/python3.10 lists.py
[1, 2, 3, 4, 4, 6]
[4, 4, 6, 1, 2, 3]

Process finished with exit code 0
```

in: mirar si un valor està a la list.

```
##Iteration
list1 = [3,9,0]

print(3 in list1)
print(6 in list1)
```

lists x

/home/roger/IJB/IJB/venv/local/bin/p
True
False

Process finished with exit code 0

indexar: es vol buscar el valor posicionat en un index concret.

```
##Indexing
list1 = [1, "hola", 3.90, [0,2,3], False]
print(list1[1])
print(list1[-3])
```

lists x

/home/roger/IJB/IJB/venv/local/bin/python3
hola
3.9

Process finished with exit code 0

Buscar valores en una **matriu**.

```
matriu1 = [[1,2.3,"hola"],[3,True,0],["Roger", 18, False]]

print(matriu1[1])
print(matriu1[0][2])
print(matriu1[0][3])
```

lists x

```
/home/roger/IJB/IJB/venv/local/bin/python3.10 /home/roger/IJB/
[3, True, 0]
hola
Traceback (most recent call last):
  File "/home/roger/IJB/IJB/M7/Teoria/PYTHON/basics/lists.py"
    print(matriu1[0][3])
IndexError: list index out of range
```

Modificar valores d'una list.

```
#Changing Lists
list1 = ["Roger", "Apple", "Tool"]
list1[2] = 'Funciona'
print(list1)
```

lists x

```
/home/roger/IJB/IJB/venv/local/bin/
['Roger', 'Apple', 'Funciona']

Process finished with exit code 0
```

append() vs extend()

```
##Changing Lists
list1 = ["Roger", "Apple", "Tool"]
list1[2] = 'Funciona'

list1.append([3,4])
print(list1)
list1.extend([5,6])
print(list1)
```

lists x

```
/home/roger/IJB/IJB/venv/local/bin/python3.10
['Roger', 'Apple', 'Funciona', [3, 4]]
['Roger', 'Apple', 'Funciona', [3, 4], 5, 6]

Process finished with exit code 0
```

sort(): ordena de forma ascendent els valors de la list.

```
##sort()
list1 = ["Roger", "Andreu", "Juan", "John", "Doe"]
list2 = [2,5,6,7,1,0,3,4]
list1.sort()
list2.sort()
print("sort de list1: ")
print(list1)
print("sort de list2: ")
print(list2)
```

lists x

```
/home/roger/IJB/IJB/venv/local/bin/python3.10 /hom
sort de list1:
['Andreu', 'Doe', 'John', 'Juan', 'Roger']
sort de list2:
[0, 1, 2, 3, 4, 5, 6, 7]

Process finished with exit code 0
```

pop(): elimina l'últim valor de la list.

```
myList = [1, 2, 3, 4]
print(myList.pop())
print(myList)

/home/roger/IJB/IJB/ve
4
[1, 2, 3]
```

Crear una list amb **for**.

```
numbers = [value**2 for value in range(1, 11)]
print(numbers)
```

```
/home/roger/IJB/IJB/venv/bin/python /home,
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

Tupla

No Son mutables (no es poden modificar).

Son una seqüència de valors indexats per números enters.

Inicialització Tupla ⇒ **myTupla = ()**

```
myTupla = (1, 2, 3, 4, 5, 6, 7)
```

len(): per conèixer la longitud de la tupla.

```
myTupla = (1, 2, 3, 4, 5, 6, 7)
print(len(myTupla))
```

```
tupla x
/home/roger/IJB/IJB/venv/bin/python
7
Process finished with exit code 0
```

in

```
x = 9
y = 3
myTupla = (1, 2, 3, 4, 5, 6, 7)
print(x in myTupla)
print(y in myTupla)
```

```
tupla x
/home/roger/IJB/IJB/venv/bin/python /t
False
True
Process finished with exit code 0
```

count(): mètode per saber les vegades que un valor es repeteix en la tupla.

```
myTupla = (1,2,"pedro",3,4,"pedro",5,6,7)
print(myTupla.count("pedro"))
```

tupla x

```
/home/roger/IJB/IJB/venv/bin/python /home/roger/IJB/IJB/M4/UF2_PYTHON/TEORIA/tupla.py
2
```

Process finished with exit code 0

Són immutables!

```
myTupla = (1,2,"pedro",3,4,"pedro",5,6,7)
myTupla[3] = "juan"
print(myTupla)
```

tupla x

```
/home/roger/IJB/IJB/venv/bin/python /home/roger/IJB/IJB/M4/UF2_PYTHON/TEORIA/tupla.py
Traceback (most recent call last):
  File "/home/roger/IJB/IJB/M4/UF2_PYTHON/TEORIA/tupla.py", line 58, in <module>
    myTupla[3] = "juan"
TypeError: 'tuple' object does not support item assignment
```

El mètode **append()** no funciona amb les tuples

```
myTupla = (1,2,"pedro",3)
myTupla.append(3)
print(myTupla)
```

tupla x

```
/home/roger/IJB/IJB/venv/bin/python /home/roger/IJB/IJB/M4/UF2_PYTHON/TEORIA/tupla.py
Traceback (most recent call last):
  File "/home/roger/IJB/IJB/M4/UF2_PYTHON/TEORIA/tupla.py", line 58, in <module>
    myTupla.append(3)
AttributeError: 'tuple' object has no attribute 'append'
```

Process finished with exit code 1

set

És una col·lecció desordenada sense elements duplicats.

Suporten operacions matemàtiques com unió, intersecció, diferència i diferència simètrica.

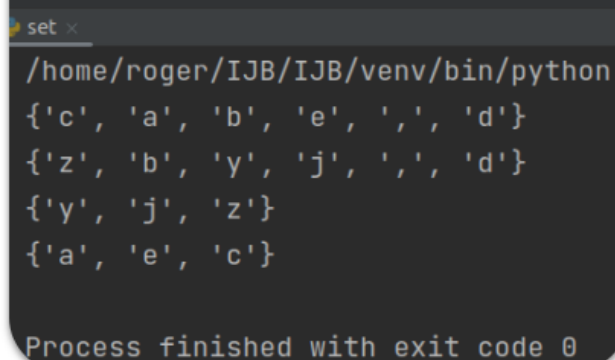
Els valors no es poden canviar, però es poden esborrar i afegir.

S'inicialitza amb $\Rightarrow$ `mySet = set()`

```
mySet = set()
```

Per buscar la diferència entre dos sets s'utilitza el símbol menys `-`.

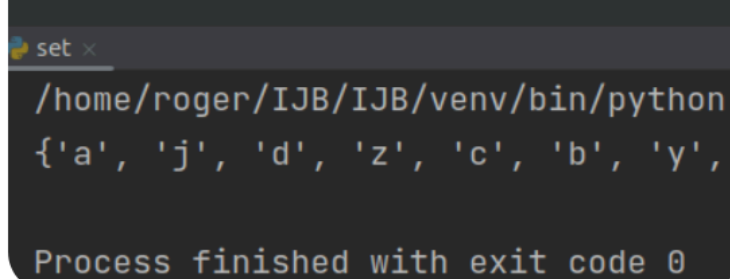
```
x = set('a,b,c,d,e')
print(x)
y = set('y,b,z,d,j')
print(y)
print(y-x)
print(x-y)
```



```
set x
/home/roger/IJB/IJB/venv/bin/python
{'c', 'a', 'b', 'e', ',', 'd'}
{'z', 'b', 'y', 'j', ',', 'd'}
{'y', 'j', 'z'}
{'a', 'e', 'c'}
Process finished with exit code 0
```

Per unir varis sets s'utilitza el símbol `|`.

```
x = set('a,b,c,d,e')
y = set('y,b,z,d,j')
print(x | y)
```



```
set x
/home/roger/IJB/IJB/venv/bin/python
{'a', 'j', 'd', 'z', 'c', 'b', 'y',
Process finished with exit code 0
```

Per saber els valors repetits s'utilitza el símbol d'intersecció &.

```
x = set('a,b,c,d,e')
y = set('y,b,z,d,j')
print(x & y)
```

```
set x
/home/roger/IJB/IJB/venv/bin/python
{'b', 'd', ' ', ' '}
Process finished with exit code 0
```

Per saber els valors no repetits s'utilitza el símbol ^.

```
x = set('a,b,c,d,e')
y = set('y,b,z,d,j')
print(x ^ y)
```

```
set x
/home/roger/IJB/IJB/venv/bin/python
{'e', 'c', 'y', 'j', 'z', 'a'}
Process finished with exit code 0
```

Per afegir valors **add()**.

```
x = set('a,b,c,d,e')
x.add('pedro')
print(x)
```

```
set x
/home/roger/IJB/IJB/venv/bin/python
{'d', 'a', 'pedro', 'c', ' ', ' ', 'e', ' '}
Process finished with exit code 0
```

remove(): per eliminar valores.

```
x = set('a,b,c,d,e')  
x.remove('c')  
print(x)
```

set x

```
/home/roger/IJB/IJB/venv/bin/python  
{'d', ' ', 'b', 'a', 'e'}
```

Process finished with exit code 0

Diccionaris

És un set desordenat que conté parells de “**key:value**”.

El contingut està indexat per “**key**”.

S’inicialitza amb $\Rightarrow$ `myDict= {}`

```
myDict = {}
```

Per afegir una **key** i el seu **value** al diccionari cal:

```
myDict = {}  
myDict["prova"] = 18  
print(myDict)
```

ictionaries x

```
/home/roger/IJB/IJB/venv/bin/python  
{'prova': 18}
```

Process finished with exit code 0

`len()`: per saber el tamany del diccionari.

```
myDict = {'prova':18, 'Roger':'Sobrino', 'Edat': 38}  
print(len(myDict))
```

ictionaries x

```
/home/roger/IJB/IJB/venv/bin/python /home/roger/IJB/IJB/  
3
```

Process finished with exit code 0

popitem() per eliminar l'últim **item** del diccionari.

```
myDict = {'prova':18, 'Roger':'Sobrino', 'Edat': 38}
print(myDict.popitem())
print(myDict)
```

```
dictionaryes x
/home/roger/IJB/IJB/venv/bin/python /home/roger/IJB/IJB,
('Edat', 38)
{'prova': 18, 'Roger': 'Sobrino'}

Process finished with exit code 0
```

del(): per eliminar contingut amb coincidència de **key**.

```
myDict = {'prova':18, 'Roger':'Sobrino', 'Edat': 38}
del myDict['Edat']
print(myDict)
```

```
dictionaryes x
/home/roger/IJB/IJB/venv/bin/python /home/roger/IJB/IJB,
{'prova': 18, 'Roger': 'Sobrino'}

Process finished with exit code 0
```

values() per saber els valors d'un diccionari.

```
myDict = {'prova':18, 'Roger':'Sobrino', 'Edat': 38}
print(myDict.values())
```

```
dictionaryes x
/home/roger/IJB/IJB/venv/bin/python /home/roger/IJB/IJB,
dict_values([18, 'Sobrino', 38])

Process finished with exit code 0
```